

06-1. Web 通识扫盲课

定位：零基础入门，为 Flask Web 开发和安全工具铺垫

核心目标：读者学完后能理解“输入网址按下回车后发生了什么”，能看懂网页源码，知道前端和后端的区别。

目录

- 06-1. Web 通识扫盲课
 - 目录
 - 第一章：Web 是什么？
 - 1.1 Web 的诞生：从一份论文到改变世界
 - 1.2 Web 的“三国时代”：浏览器大战
 - 1.3 你每天都在用 Web，但你知道它是什么吗？
 - 1.4 前端和后端有什么区别？
 - 第二章：HTML——网页的骨架
 - 2.1 HTML 是什么？
 - 2.2 HTML 的基本结构
 - 2.3 HTML 表单——用户输入的第一道门
 - 第三章：CSS——网页的皮肤
 - 3.1 CSS 是什么？
 - 3.2 CSS 的基本语法
 - 3.3 CSS 和 HTML 的关系
 - 第四章：JavaScript——网页的大脑
 - 4.1 JavaScript 是什么？
 - 4.2 JavaScript 的基本语法
 - 4.3 JavaScript 被恶意利用：XSS 的原理
 - 第五章：HTTP 协议——Web 的语言
 - 5.1 HTTP 是什么？
 - 5.2 常见的 HTTP 方法
 - 5.3 常见的 HTTP 状态码
 - 5.4 Cookie 和 Session
 - 第六章：用开发者工具透视网页——你的第一把“透视眼”
 - 6.1 从 HTML 到像素——浏览器做了什么？
 - 6.2 浏览器的开发者工具——你的第一把“透视眼”
 - 第七章：总结与展望
 - 7.1 你学会了什么？
 - 7.2 这些知识在渗透测试中怎么用？
 - 7.3 下一步学什么？

第一章：Web 是什么？

Kate 在 05 的第四章用 Wireshark 亲手抓过 HTTP 请求和响应，在 05 的第二章用 Burp Suite 改过包、重放过请求。但她今天问了一个很根本的问题：

“我每天都在用浏览器看网页——输入网址、按回车、页面出来了。我知道背后有 HTTP、有 HTML、有服务器。但 Web 这个东西到底是怎么来的？谁发明的？为什么叫‘Web’？”

“这个问题问到了 Web 的老祖宗。故事要从 1989 年的一份论文提案讲起。”

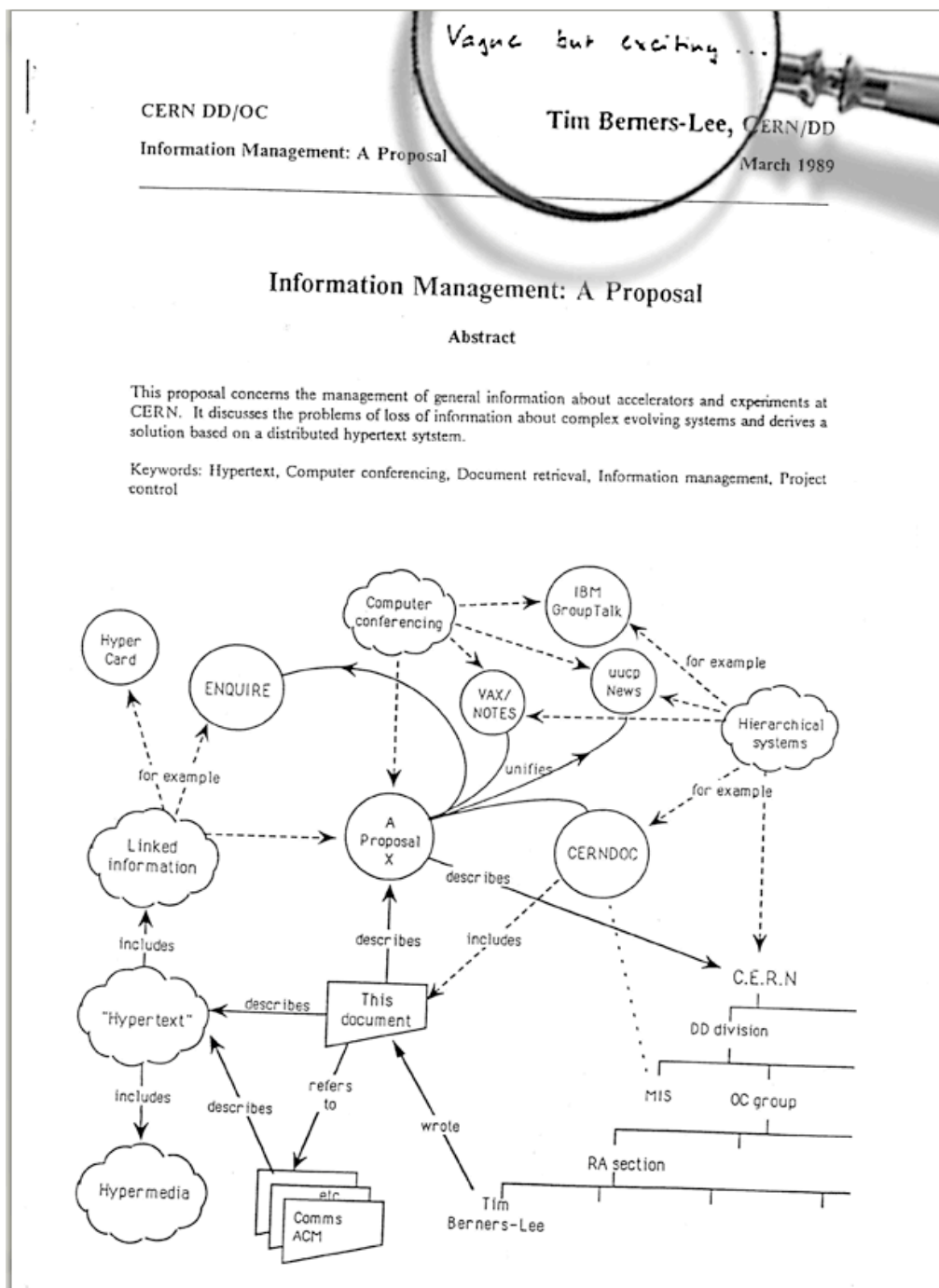
1.1 Web 的诞生：从一份论文到改变世界

1989 年，一个叫 **Tim Berners-Lee** 的英国软件工程师在欧洲核子研究中心（CERN）工作。CERN 是世界上最大的粒子物理实验室——来自几十个国家的上万名科学家在这里合作研究。但 Berners-Lee 发现了一个极其恼人的问题：**每个人的文档都在自己的电脑上，不同电脑上的文档互相看不到。**

你想看同事的研究报告，得先知道他的电脑名、登录他的系统、找到文件路径、用对应的软件打开。每换一台电脑、每换一种文件格式，这套流程就要重新来一遍。科学家把大量时间浪费在了“找文件”上，而不是“做研究”。

Berners-Lee 想了一个方案：**能不能让所有文档互相链接？**就像学术论文里的脚注——你读到一段引文，脚注告诉你出处，你翻到那一页就能看到原文。如果把这种“脚注”搬到电脑上——你点击文档里的一个链接，它直接跳转到另一个文档，不需要知道那个文档在哪台电脑上——信息就真正连成了一张网。

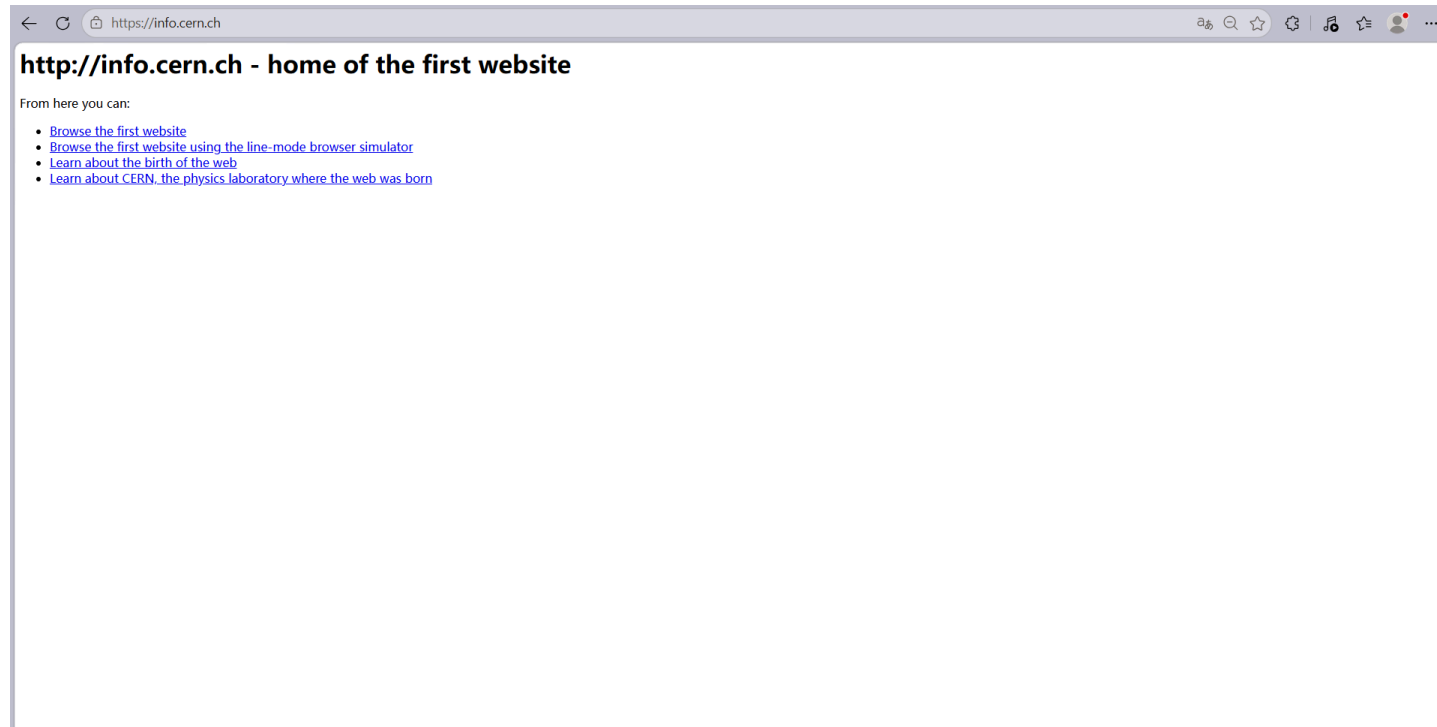
1989 年 3 月，Berners-Lee 写了一份提案，标题很朴素：《信息管理：一个提案》（*Information Management: A Proposal*）。他的老板在提案封面写了一句话作为批复：“模糊但令人兴奋。”（*Vague but exciting.*）就这一句话，他拿到了继续开发的许可。



1990 年，Berners-Lee 发明了三样东西，定义了 Web 的基石：

- **HTML** (HyperText Markup Language)：用来写网页的标记语言。它让你在文档里嵌入链接——点击一段文字，跳转到另一个文档。你在 01 学的 Markdown，本质上就是 HTML 的简化版——Markdown 用 # 表示标题，HTML 用 <h1> 表示标题。
- **URI** (Uniform Resource Identifier，后来叫 URL)：给每个网页一个唯一地址。就像每家每户都有一个门牌号，每个网页也有一个独一无二的地址——`http://info.cern.ch`。你在 05-01 第四章学的 IP 地址和 DNS，就是用来解析这些 URL 的。
- **HTTP** (HyperText Transfer Protocol)：浏览器和服务器之间传输网页的协议。你在 05-01 第四章学的 HTTP 请求和响应、GET 和 POST、200 OK 和 404 Not Found——全都是 Berners-Lee 在 1990 年定义的这套规则。

1991 年，世界上第一个网站上线：<http://info.cern.ch>。这个网站现在还活着，你可以在浏览器里打开它——它只包含文字和链接，没有图片、没有动画、没有交互。但它实现了一件以前从未有过的事：**你在一台电脑上点击一个链接，看到了另一台电脑上的文档。**



Web 这个名字是怎么来的？Berners-Lee 的原意是：**World Wide Web——一张覆盖全球的蜘蛛网**。每个网页是一个节点，链接是连接节点的丝线。你点击链接，就像蜘蛛在网上爬行——从一个节点跳到另一个节点，从一台电脑跳到另一台电脑。整个互联网就是一张巨大的、由链接编织而成的网。

Kate 在浏览器里打开了 <http://info.cern.ch>，看到了那个只有文字和链接的黑白页面。她说：“这就是世界上第一个网站？它连一张图片都没有——但它是所有网页的祖宗。我今天打开的 GitHub、知乎、B 站，底层都是这三样东西。”

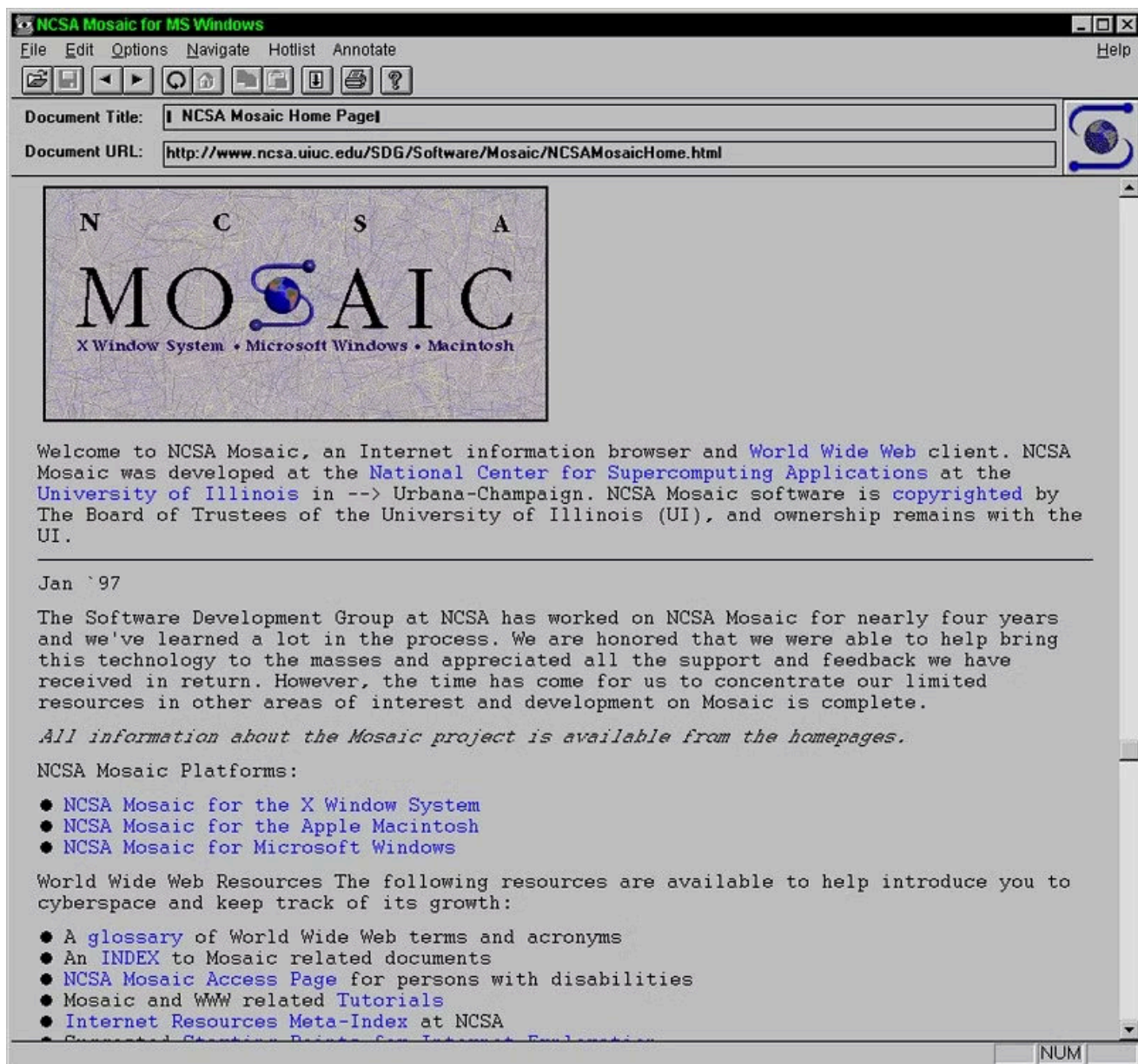
“对。而且你以后学的所有 Web 渗透测试技术——SQL 注入、XSS、CSRF、文件上传——全都是利用了这三样东西的某些特性。你不理解 HTML、URL、HTTP，你就理解不了为什么一段 `<script>` 标签能让浏览器弹窗，为什么在 URL 后面加一个单引号能让数据库报错。”

1.2 Web 的“三国时代”：浏览器大战

Berners-Lee 发明了 Web，但 Web 要走进千家万户，需要一件东西——**浏览器**。没有浏览器，你看到的就是一堆 HTML 源码，而不是一个渲染好的网页。

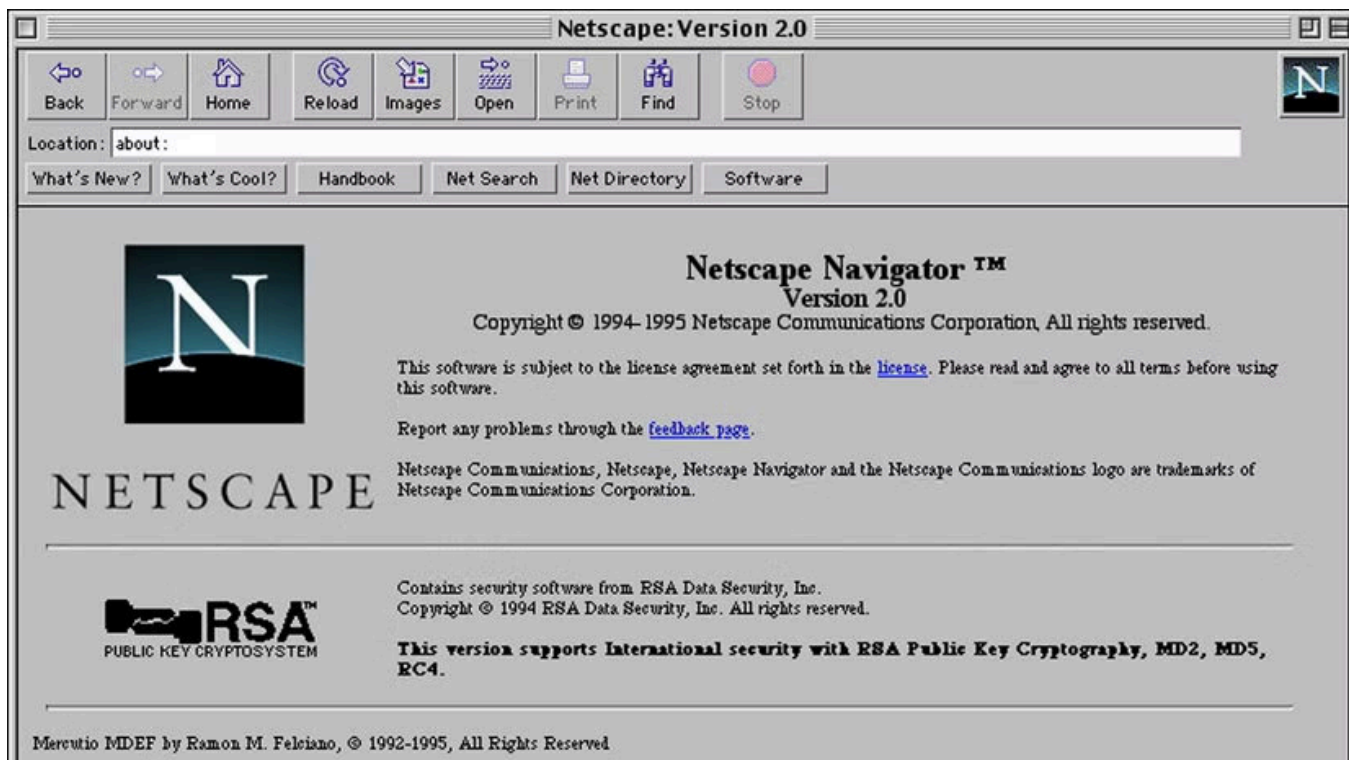
世界上第一个浏览器是 Berners-Lee 自己写的，叫 **WorldWideWeb**（后来改名叫 Nexus）。它既能浏览网页，也能编辑网页——Berners-Lee 的本意是“Web 是双向的，每个人既是读者也是作者”。但这个想法太超前了，后来的浏览器几乎全部砍掉了编辑功能，只保留了浏览。

真正让 Web 火起来的，是 1993 年发布的 **Mosaic** 浏览器。Mosaic 由美国伊利诺伊大学的国家超级计算应用中心（NCSA）开发。它做了三件革命性的事：首次支持在文字中**内嵌显示图片**（在此之前，图片必须在新窗口打开），首次引入了**图形化用户界面**——书签、后退按钮、地址栏，普通人不需要学命令就能用。而且它是**免费的**，任何人都可以下载使用。



Mosaic 一发布就炸了。1993 年初，全球只有不到 100 个网站。1993 年底 Mosaic 发布后，网站数量开始指数级增长。普通人才知道——原来互联网可以看图片，可以点链接跳来跳去，而且不用学任何技术。Mosaic 把 Web 从“科学家和极客的玩具”变成了“所有人都能用的工具”。

Mosaic 的核心开发者 **Marc Andreessen** 后来离开 NCSA，创办了 **Netscape** 公司，发布了 **Netscape Navigator** 浏览器。Netscape 继承了 Mosaic 的所有优点，加了更多功能——JavaScript（让网页能动起来）、Cookie（让网站记住你是谁）、SSL 加密（让网上购物成为可能）。到 1995 年，Netscape 占据了超过 80% 的浏览器市场份额。



这时候，微软坐不住了。比尔·盖茨在 1995 年发了一份内部备忘录，标题叫《互联网浪潮》（*The Internet Tidal Wave*）。他在备忘录里写道：“互联网是自 IBM PC 以来最重要的技术变革。”然后微软做了三件事：从 Spyglass 公司购买了 Mosaic 的源代码（是的，Mosaic 的代码被卖给了微软），开发了自己的浏览器 **Internet Explorer (IE)**，把 IE 捆绑在 Windows 操作系统中**免费赠送**。

这就是著名的**第一次浏览器大战**（Browser Wars I）。Netscape 收费（个人用户免费，企业收费），微软全免费。Netscape 要自己开发所有功能，微软可以直接利用 Windows 的底层接口。Netscape 是一个独立的浏览器，IE 是 Windows 的一部分——你买一台 Windows 电脑，IE 已经在桌面上了。Netscape 没有任何优势。

到 1998 年，IE 的市场份额超过 90%。Netscape 被打败了，1999 年被 AOL 收购。但 Netscape 在倒下之前做了一件事：**把浏览器内核的源代码全部公开**。这个开源项目叫 **Mozilla**——它后来演变成了 **Firefox** 浏览器。Netscape 虽然死了，但它的代码活了下来。

微软在打败 Netscape 之后，把 IE 团队裁撤到只剩几个人，浏览器功能几乎停止更新。IE 6 在 2001 年发布后，整整五年没有大版本更新。而 Web 技术在飞速发展——新的 HTML 标签、新的 CSS 属性、新的 JavaScript API。IE 6 对这些新特性的支持极其糟糕，全球的 Web 开发者每天都在咒骂微软。这段时期被开发者称为“IE 6 的黑暗时代”。



2004 年，Mozilla 发布了 **Firefox**——它是 Netscape 开源代码的后代，但完全重写了渲染引擎。Firefox 快速、安全、支持最新的 Web 标准，一夜之间席卷全球。2008 年，Google 发布了 **Chrome**——它更简洁、更快，每个标签页独立运行，一个标签页崩溃不会影响其他标签页。Chrome 还首次引入了 **V8 JavaScript 引擎**，让 JavaScript 的执行速度比以前快了好几倍。Web 从“展示文档的工具”变成了“运行应用的平台”——你在线编辑的 Google 文档、你玩的网页游戏、你用的 Figma 设计工具，全是 Chrome 强大的 JavaScript 引擎撑起来的。

到 2012 年，Chrome 的市场份额超过 IE。微软宣布 IE 退役，换成全新的 **Edge** 浏览器。2020 年，微软干脆放弃了自己的浏览器内核，Edge 改用 Chrome 的内核。**当年的浏览器大战，赢家不是微软，也不是 Netscape——是 Chrome。**

Kate 说：“所以 Netscape 打输了，但它开源了代码，后来变成了 Firefox。微软的 IE 打赢了，但后来又被 Chrome 打败，连自己的内核都放弃了。Chrome 赢了，靠的是更快、更安全、更支持 Web 标准。”

“对。浏览器大战不是一场‘谁更狠’的战争，是‘谁更开放、谁更符合标准’的战争。Netscape 失败了，但它的开源精神活了下来。IE 打赢了战争，但它在技术上的落后让它最终死掉了。Chrome 赢在它一直在追赶 Web 标准的脚步——你以后写 HTML、CSS、JavaScript 的时候，Chrome 能按照标准把你的代码渲染出来。”

为什么学安全要了解浏览器大战？

你在 05 学过的很多攻击技术，都和浏览器的发展历史直接相关。XSS 攻击能在 JavaScript 出现之后才成为一种普遍的威胁——没有 JavaScript，就没有人能劫持网页逻辑。Cookie 被 Netscape 发明用来维持登录状态，它后来成为所有 Web 应用认证的基础，也成为 XSS 攻击最想窃取的目标。SSL/TLS 加密最早是为了保护电子商务交易，它后来发展成了 HTTPS——你在 05 第四章学的 HTTP 明文传输的弱点，就是 HTTPS 试图解决的问题。IE 6 的兼容性问题导致很多遗留漏洞——你以后在内网渗透中还会遇到还在运行 IE 6 的老旧系统，它们无法修复现代安全补丁。

你不需要记住这场战争的全部细节。你只需要知道一件事：你今天用的 Chrome、Firefox、Edge，以及你在 05 学过的 XSS、Cookie、HTTPS——所有这些，全都是这三十年浏览器战争的直接遗产。

Kate 说：“所以 Netscape 虽然死了，但它留下的 JavaScript、Cookie、SSL——这些东西我今天还在用。我在 05 第二章弹的那个 XSS 弹窗，用的是 JavaScript。我在 05 第四章抓的 HTTP 请求，底层用的是 HTTP 和 SSL。我以为我在学安全——原来我在学三十年战争留下来的遗产。”

1.3 你每天都在用 Web，但你知道它是什么吗？

Kate 在上一节听完了浏览器大战的历史，靠在椅背上想了一会儿，然后说：

“我每天打开浏览器、输入网址、按回车——网页就出来了。但仔细一想，我好像根本不知道这个过程中发生了什么。‘浏览器’是什么？‘服务器’在哪？我输入的‘网址’到底是什么东西？”

“这个问题就是这一节要回答的。你每天用的 Web，其实只有三个角色在互相配合。你把这三个角色搞清楚，Web 的基本原理你就全懂了。”

Web 的三大角色

你在浏览器里看到一个网页，背后其实是三个“角色”在协同工作：

角色	是什么	你每天在用的例子	它在 Web 中的职责
客户端	你用来上网的软件	Chrome、Edge、Firefox、Safari	发起请求，渲染网页给你看
服务器	存放网页的远程电脑	GitHub 的服务器、B 站的服务器	接收请求，返回网页数据
HTTP 协议	客户端和服务端之间的“通信语言”	你在 05 第四章学过的 HTTP	规定请求和响应的格式

这三个角色的关系可以用一句话总结：**客户端通过 HTTP 协议向服务器发起请求，服务器通过 HTTP 协议返回响应。**

“客户端”这个名字听起来很高端，但它的意思很简单：**你是客户，浏览器替你跑腿。**你想看 GitHub 上的某个仓库，你不需要自己跑到 GitHub 的服务器机房去敲门——你只需要在浏览器的地址栏里输入网址，浏览器会替你发请求、收响应、把网页渲染在屏幕上。浏览器是你的“代理人”。

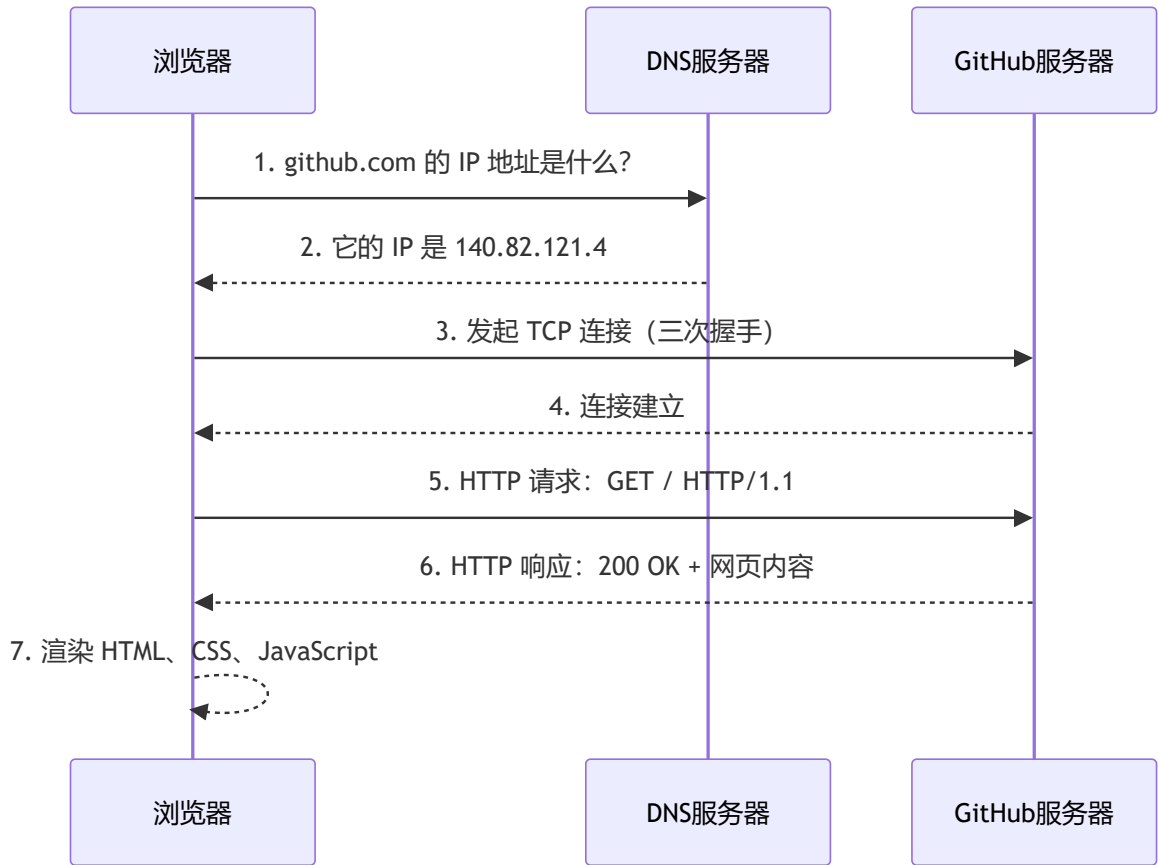
“服务器”是一台远在千里之外的电脑。它和你的个人电脑没有本质区别——也有 CPU、内存、硬盘、操作系统。区别在于它安装的是服务器软件（而不是你用的 Windows 桌面），而且它永远不关机。你凌晨三点想访问 GitHub，GitHub 的服务器必须还在运行——它不能和你一样到点睡觉。

“HTTP 协议”是客户端和服务端之间的通信语言。你在 05 第四章已经学过 HTTP 的请求和响应格式——请求行、请求头、请求体、状态码。客户端和服务端必须说同一种语言，否则它们无法理解对方在说什么。

Kate 说：“所以 Web 就是一个‘传话游戏’。我是客户，浏览器替我传话。服务器在另一头听着，收到我的请求之后，把网页数据传回来。HTTP 就是我们之间的传话规则——每一句话的格式都是规定好的。”

一个完整的网页请求流程

你输入 `https://github.com`，按下回车。在这个瞬间，你的浏览器做了以下五件事：



第一步：DNS 解析——把名字翻译成门牌号

你输入的是 `github.com`，但服务器只认识 IP 地址。浏览器做的第一件事，就是把域名翻译成 IP 地址。这一步叫 DNS 解析，你在 05 第四章已经详细学过 DNS 的查询流程——浏览器先查自己的缓存，再查操作系统缓存，再问 DNS 服务器，最终拿到 `140.82.121.4` 这个 IP 地址。

第二步：TCP 连接——双方确认可以通话

浏览器拿到 IP 地址后，向 GitHub 服务器发起 TCP 连接。这一步就是你在 05 学过的 TCP 三次握手——客户端发 SYN，服务器回 SYN+ACK，客户端再回 ACK。三次握手完成后，双方确认可以开始传输数据。

第三步：HTTP 请求——浏览器说“我要这个页面”

TCP 连接建立后，浏览器发出 HTTP 请求。这个请求是明文的（如果用的是 HTTP）或者加密的（如果用的是 HTTPS）。它的大致内容是：`GET / HTTP/1.1`，表示“我要获取这个网站的首页”。请求里还附带了一些额外的信息——你用的浏览器类型、你能接收的文件格式、你之前是否登录过（Cookie）。

第四步：HTTP 响应——服务器返回网页内容

GitHub 服务器收到请求后，找到首页对应的文件，把它打包进 HTTP 响应里返回给浏览器。响应的第一行是状态码——`HTTP/1.1 200 OK`，表示“请求成功”。响应头里包含了网页内容类型（`text/html`）和长度。响应体里就是你要看的网页内容——HTML 代码。

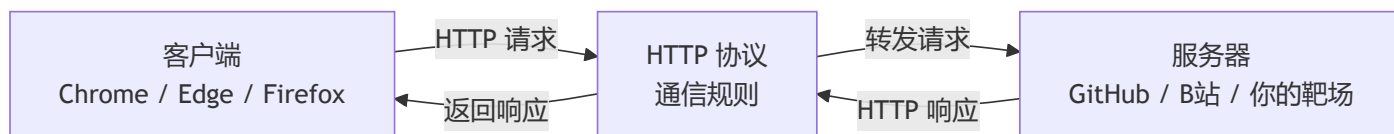
第五步：浏览器渲染——把代码变成你看到的网页

浏览器收到 HTML 代码后，开始解析它——构建 DOM 树、加载 CSS、执行 JavaScript。最终，一堆 `<div>`、`<p>`、`<a>` 标签被渲染成了你看到的那个漂亮的 GitHub 首页。这个过程你在 6.1 节会深入理解。

Kate 说：“所以我在地址栏输入 `github.com`，按下回车——这背后是 DNS 查 IP、TCP 三次握手、HTTP 请求和响应、浏览器渲染。我以为是一瞬间的事，其实是有五个步骤在排队执行。”

“对。而且你在 05 已经亲手验证过其中的好几个步骤——用 Wireshark 抓过 DNS 查询、看过 TCP 三次握手的 SYN 和 ACK 包、分析过 HTTP 请求的 GET 和 200 OK。现在你把它们串起来了。”

浏览器、服务器、HTTP 的关系：一张图总结



Kate 看着这张图，说：“所以我就是客户端，浏览器是我的代理。HTTP 是传话规则，服务器是传话对象。三样东西，组成了整个 Web 世界。”

1.4 前端和后端有什么区别？

- **前端**：你在浏览器里看到的一切——按钮、输入框、动画、颜色
- **后端**：在服务器上运行的程序——处理登录、存取数据库、返回数据
- 前后端如何通过 HTTP 通信：请求和响应
- 为什么学安全要理解前后端？因为前端有 XSS 和 CSRF，后端有 SQL 注入和文件上传——漏洞藏在每一层里

第二章：HTML——网页的骨架

Kate 在第一章看完了 Web 的诞生和浏览器大战。她靠在椅背上，说了一句：

“Web 的历史我懂了。但我还有一个问题——网页本身是用什么写的？我右键点开一个网页，选‘查看源代码’，跳出来一堆尖括号和英文单词——那是什么？”

“那就是 HTML。你今天看到的所有网页——GitHub、B 站、知乎——全都是用它写的。它不是编程语言，但它是整个 Web 的骨架。”

2.1 HTML 是什么？

HTML 的全称是 **HyperText Markup Language**——超文本标记语言。这个名字拆开来，每个词都有含义：

- **HyperText**（超文本）：不只是普通文字——文字里可以嵌入链接，点击链接就能跳到另一个页面。你在 1.1 节学过，这是 Tim Berners-Lee 发明 Web 时最核心的想法。没有超文本，Web 就不叫 Web——它只是一堆孤立的文档，彼此没有联系
- **Markup**（标记）：你在文字外面套一层“标签”，告诉浏览器这段文字应该怎么显示。`<h1>` 是“这是一级标题”，`<p>` 是“这是一个段落”，`<a>` 是“这是一个链接”
- **Language**（语言）：HTML 是一套语法规则——哪些标签是合法的、标签之间怎么嵌套、属性怎么写。浏览器按照这套规则解析 HTML，把它渲染成你看到的网页

HTML 不是编程语言。

编程语言（比如你以后要学的 Python）有变量、有循环、有逻辑判断——它们能“计算”。标记语言只有标签——它们只能“描述”。HTML 不能算 1+1，不能判断“如果用户没登录就跳转到登录页”，不能从数据库里读取数据。它只做一件事：**描述网页的结构**。这一段是标题，那一段是段落，这个字要加粗，那个字是链接。

Kate 说：“所以 HTML 就像建筑工地的脚手架——它决定了网页的结构，哪里是标题、哪里是段落、哪里是图片。它不负责好看（那是 CSS 的事），也不负责交互（那是 JavaScript 的事）。它只管结构。”

“精准。而且这个脚手架，你在学 Markdown 的时候已经见过一个简化版了。”

HTML 和 Markdown 的关系

你在 01 学的 Markdown，本质上就是 HTML 的简化版。Markdown 用 `#` 表示标题，HTML 用 `<h1>` 表示标题。Markdown 用 `**文字**` 表示加粗，HTML 用 `<strong>文字</strong>` 表示加粗。Markdown 用 `-` 表示列表，HTML 用 `<ul><li>` 表示列表。

这不是巧合——Markdown 的设计目标就是“用最简洁的语法写 HTML”。你在 01 用 Markdown 写的每一篇笔记、每一章教程，渲染引擎在背后都把它们翻译成了 HTML，然后浏览器才能显示。你写 `# 标题`，渲染引擎把它翻译成 `<h1>标题</h1>`。你写 `**加粗**`，渲染引擎把它翻译成 `<strong>加粗</strong>`。

那为什么不直接用 Markdown 写网页？因为 Markdown 的功能太少了。它只有标题、段落、列表、链接、图片、代码块——你没法用它画一个登录表单，没法用它做一个导航栏，没法用它嵌入视频和交互式图表。Markdown 适合写文档——你正在读的这套教程就是用它写的。HTML 适合写网页——你用浏览器打开的每一个网站都是用它写的。

Kate 说：“所以 Markdown 是 HTML 的‘省事版’——写文档的时候用 Markdown，因为只需要标题、段落、列表。写网页的时候用 HTML，因为需要表单、导航栏、视频播放器。底层是同一套东西，只是语法不同。”

亲手写你的第一个 HTML 页面

打开 VS Code（或者 Vim），新建一个文件，命名为 `my-first-page.html`。把下面这段代码敲进去（不要复制粘贴，亲手敲）：

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>守夜者的第一个网页</title>
</head>
<body>
  <h1>欢迎来到守夜者的网站</h1>
  <p>这是我的第一个 HTML 页面。</p>
  <p>HTML 不是编程语言——它是<strong>标记语言</strong>。</p>
</body>
</html>
```

保存后，双击这个文件，它会在你的浏览器里打开。你会看到一个网页——标题是“守夜者的第一个网页”，正文有两段文字，其中“标记语言”四个字是加粗的。

你现在看到的这个网页，就是 HTML 最基础的样子。它不漂亮——白底黑字，没有任何颜色和布局。但它是一个结构完整的网页：有大标题（`<h1>`），有段落（`<p>`），有加粗（`<strong>`）。后面的第三章会教你怎么用 CSS 让它变漂亮，第四章会教你怎么用 JavaScript 让它变聪明。但现在，你只需要理解骨架。

欢迎来到守夜者的网站

这是我的第一个 HTML 页面。
HTML 不是编程语言——它是**标记语言**。

Kate 敲完这段代码，双击打开，看到了自己人生中写的第一个网页。她说：“这就是一个网页？就这么几行？我以为网页是那种——几千行代码、很复杂的东西。”

“几千行的网页也只是这几行代码的扩展。你看到的那个漂亮的 GitHub 首页，底层也是 `<h1>`、`<p>`、`<a>` 这些标签——只是数量更多、嵌套更复杂。所有网页的骨骼，都是你现在看到的这六样标签搭起来的。”

2.2 HTML 的基本结构

Kate 在上一节亲手写了人生中第一个 HTML 页面。她盯着自己写的几行代码，问了一个问题：

“`<!DOCTYPE html>`、`<html>`、`<head>`、`<body>`——这些标签是干什么的？每个网页都必须有它们吗？为什么我写的文字要放在 `<body>` 里面？”

“这些就是 HTML 的**骨架标签**——每个网页都从它们开始。你不需要背，但你需要知道它们各自负责什么。”

每个网页都有的四个骨架标签

你在上一节写的那个页面，去掉文字内容之后，核心结构是这样的：

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>守夜者的第一个网页</title>
</head>
<body>
  <!-- 这里放网页的实际内容 -->
</body>
</html>
```

这五个标签（`<!DOCTYPE>`、`<html>`、`<head>`、`<meta>`、`<body>`）是每个网页都必须有的骨架。它们不负责显示内容——它们负责告诉浏览器“这是什么类型的文档、用什么语言、用什么编码、实际内容从哪开始”。

标签	作用	比喻
<code><!DOCTYPE html></code>	告诉浏览器“这是一份 HTML5 文档”	快递单上印的“文件类型：文档”
<code><html></code>	整个网页的根标签，所有内容都在它里面	整个快递包裹的外包装
<code><head></code>	网页的“元信息”——标题、编码、关键词。 不在页面上显示	快递单上贴的标签
<code><meta charset="UTF-8"></code>	告诉浏览器“用 UTF-8 编码解析这份文档”	告诉快递员“这个包裹上的字是用中文写的，别拿英文的读法去读”
<code><body></code>	网页的实际内容——你看到的所有文字、图片、按钮都在这里	包裹里面实际装的东西

Kate 说：“所以 `<head>` 是给浏览器看的——标题、编码——但用户看不到。`<body>` 是给用户看的——文字、图片、按钮——我在页面上看到的一切都在 `<body>` 里。”

`<meta charset="UTF-8">` 是干什么的？

`<meta charset="UTF-8">` 这行代码放在 `<head>` 标签内部，用来告诉浏览器“请用 UTF-8 编码来解析这个网页”。

如果缺少这行代码，浏览器可能使用错误的编码来解析你的网页——当你用中文字符时，页面可能会变成一团乱码。这就像你给一个不懂中文的人发了一段中文消息，但没有告诉他这是中文——他可能会尝试用英文、法文、俄文的读法去理解，结果什么都读不出来。

UTF-8 是目前互联网上使用最广泛的编码方式，它覆盖了所有国家的文字——英文、中文、日文、阿拉伯文、甚至你以后可能会用到的表情符号。你不需要深入理解编码的原理，但你写的每一个 HTML 文件都应该在 `<head>` 标签内部加上这一行代码。养成习惯，不要省略。

常用标签：搭建网页的基本积木

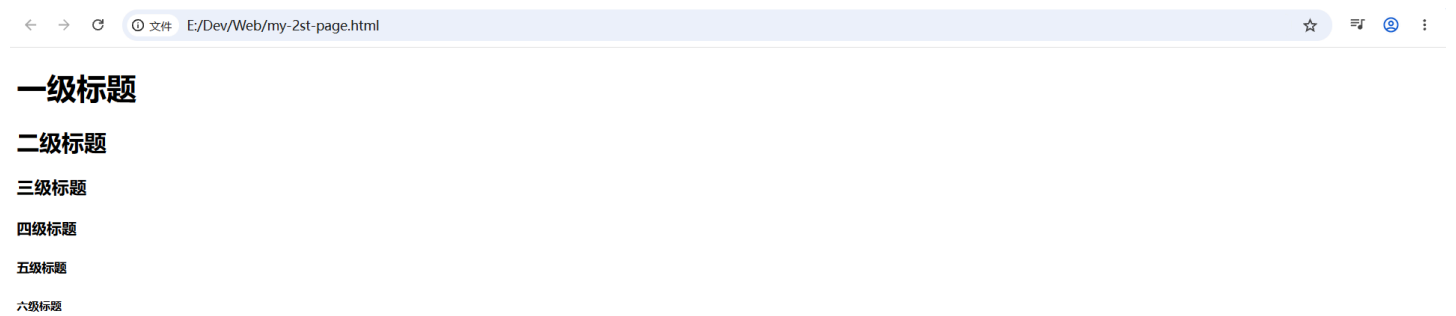
HTML 提供了几十种标签，但你只需要记住几个最常用的。它们就像乐高积木的基础块——你不需要拥有所有形状，只要有这几块，就能搭出完整的结构。

标题：`<h1>` 到 `<h6>`

```
<h1>一级标题</h1>
<h2>二级标题</h2>
<h3>三级标题</h3>
<h4>四级标题</h4>
<h5>五级标题</h5>
<h6>六级标题</h6>
```

标题标签从 `<h1>`（最大）到 `<h6>`（最小）。你在 01 学的 Markdown 标题——`#` 到 `#####`——对应的就是这六个标签。你写 `# 一级标题`，Markdown 渲染引擎在背后把它翻译成了 `<h1>一级标题</h1>`。现在你直接写 HTML，不需要渲染引擎翻译

了——浏览器直接认识 `<h1>`。



段落： `<p>`

```
<p>这是一段文字。</p>
<p>这是另一段文字。</p>
```

`<p>` 是 *paragraph* 的缩写。每个 `<p>` 标签之间会自动换行并留出间距。你在 Markdown 里写的每一段文字——两个回车之间的内容——都会被翻译成 `<p>` 标签。

链接： `<a>`

```
<a href="https://github.com">访问 GitHub</a>
```

`<a>` 是 anchor（锚点）的缩写。`href` 属性指定点击后跳转的地址。你在 Markdown 里写的 `[GitHub](https://github.com)` 就会被翻译成 `<a href="https://github.com">GitHub</a>`。

`<a>` 标签不止可以跳转到其他网站——它还可以跳转到同一个页面内的另一个位置。你以后在做渗透测试的时候，经常会在 HTML 源码里看到大量的 `<a>` 标签——每一个链接都是一个可能的攻击入口。

图片： `<img>`

```

```

`<img>` 是 image 的缩写。`src` 属性指定图片的路径，`alt` 属性指定图片加载失败时显示的替代文字。你在 Markdown 里写的 `![图片描述](图片地址)` 就会被翻译成 ``。

无序列表： `<ul>` + `<li>`

```
<ul>
  <li>苹果</li>
  <li>香蕉</li>
  <li>橘子</li>
</ul>
```

`<ul>` 是 *unordered list*（无序列表）的缩写，`<li>` 是 list item（列表项）的缩写。你在 Markdown 里写的
- 苹果、 - 香蕉、 - 橘子 就会被翻译成上面的 HTML 结构。



有序列表： `<ol>` + `<li>`

```
<ol>
  <li>第一步</li>
  <li>第二步</li>
  <li>第三步</li>
</ol>
```

`<ol>` 是 ordered list（有序列表）的缩写。你在 Markdown 里写的 1. 第一步、 2. 第二步、 3. 第三步 就会被翻译成上面的 HTML 结构。

Kate 敲完了所有的示例标签，在浏览器里看到了标题、段落、链接、图片、列表。她说：“所以 Markdown 的每一个语法，背后都对应一个 HTML 标签。# 是 `<h1>`，** 是 `<strong>`，- 是 `<ul><li>`。我以前写 Markdown 的时候，以为自己是直接在写排版——其实我是在写 HTML，只是用了更简单的语法。”

“对。Markdown 是披着简单外衣的 HTML。你现在直接写 HTML，就等于把 Markdown 的外衣脱掉了——你看到的是 HTML 的原貌。”

2.3 HTML 表单——用户输入的第一道门

Kate 在上一节学会了 HTML 的基本标签——标题、段落、链接、图片、列表。她看着自己写的那个简单网页，提了一个问题：

“这些标签都是用来展示内容的。但如果我想让用户输入东西呢？比如登录框、搜索框、评论区——这些是怎么写的？”

“那就需要 HTML 表单。你在 05 学的 SQL 注入，攻击的就是表单。你在 05 学的 XSS，攻击的也是表单。表单是用户和服务器之间的第一道门——也是你以后挖漏洞的入口。”

表单是什么？

表单（Form）是 HTML 里用来收集用户输入的一组标签。你在网上见到的所有输入框、密码框、搜索框、下拉菜单、复选框、提交按钮——全都是表单的一部分。

表单的工作流程只有三步：用户在浏览器里填写表单、点击提交按钮、浏览器把用户填写的数据发送给服务器。服务器收到数据后，处理它——比如验证用户名和密码、把搜索关键词发给搜索引擎、把评论存入数据库。

一个最简单的表单长这样：

```
<form action="/login" method="POST">
  <label for="username">用户名: </label>
  <input type="text" id="username" name="username">

  <label for="password">密码: </label>
  <input type="password" id="password" name="password">

  <button type="submit">登录</button>
</form>
```

这段代码在浏览器里会渲染成两个输入框（一个文本框，一个密码框）和一个登录按钮。



表单的核心标签

标签	作用	常用属性
<form>	表单的容器， 定义数据提交到哪里、	action （提交地址）、 method （GET 或 POST）

标签	作用	常用属性
	用什么方式提交	
<input>	输入控件——文本框、密码框、单选按钮、复选框	type（控件类型）、name（数据名称）、placeholder（提示文字）
<label>	输入框的标签， 点击标签会自动聚焦到对应输入框	for（关联的输入框 id）
<button>	提交按钮	type="submit" 表示点击后提交表单

<input> 的常见类型

<input> 是表单里最灵活的标签——通过 type 属性，它能变成完全不同的控件：

type 值	外观	用途	用户输入的内容
text	单行文本框	用户名、搜索关键词	明文可见
password	密码框（输入字符显示为圆点）	密码输入	明文可见（输入时）、圆点显示
email	文本框（带邮箱格式验证）	电子邮箱地址	明文可见
number	数字选择器	年龄、数量	数字
checkbox	复选框	多选选项	勾选/未勾选
radio	单选按钮	单选选项	选中/未选中
hidden	不可见	传递不需要用户看到的额外数据	隐藏文本

Kate 看着 type 列表，说：“所以 <input> 就像一个百变怪——改一下 type 属性，它就变成完全不同的输入方式。文本框、密码框、复选框、单选按钮——全是一个标签的变种。”

GET 和 POST：表单提交的两种方式

<form> 标签有一个 method 属性，它决定表单数据以什么方式发送给服务器。只有两种选择：**GET** 和 **POST**。

GET——数据拼在 URL 后面

当 method="GET" 时，浏览器把表单数据拼接在 URL 的末尾，用 ? 分隔。你在搜索引擎里输入“守夜者之书”，点击搜索——浏览器跳转到的 URL 可能是：

https://www.google.com/search?q=守夜者之书

? 后面的 q=守夜者之书 就是你输入的表单数据。GET 请求的优点是可以被收藏、分享、在浏览器历史里找到。缺点是数据暴露在 URL 里——不适合传输密码、身份证号等敏感信息。而且 URL 有长度限制，不适合传输大量数据。

POST——数据放在请求体里

当 method="POST" 时，浏览器把表单数据放在 HTTP 请求体里发送给服务器。用户在登录框里输入用户名和密码，点击登录——浏览器发出去的请求：

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

username=kate&password=123456
```

用户输入的密码不在 URL 里，在请求体里。POST 请求不会被浏览器自动缓存、不会显示在地址栏、不会出现在浏览历史里。适合传输敏感信息、大量数据、文件上传。但 POST 不意味着安全——数据在网络上仍然是明文传输的（除非使用 HTTPS），只是不暴露在 URL 里而已。

对比维度	GET	POST
数据位置	URL 后面的 <code>?</code> 参数中	HTTP 请求体内
可见性	地址栏可见	地址栏不可见
数据量限制	URL 长度限制（约 2048 字符）	理论上无限制
可否收藏/分享	 可以	 不可以
浏览器后退	无害——重新请求同一个 URL	浏览器会提示“是否重新提交表单”
适用场景	搜索、筛选、翻页	登录、注册、修改密码、上传文件

Kate 说：“所以 GET 就像明信片——内容写在地址栏下面，谁都能看到。POST 就像密封的信封——内容藏在信封里，路过的人看不到，但路上还是可能被拆开（除非加了 HTTPS 加密）。”

表单和安全：你以后挖漏洞的入口

你在 05 学的 SQL 注入，攻击的就是表单。一个登录框，用户名输入 `' OR 1=1 --`，密码随便填——如果后端代码没有过滤用户输入，SQL 语句就会变成：

```
SELECT * FROM users WHERE username=' ' OR 1=1 -- ' AND password='xxx'
```

`OR 1=1` 永远为真，`--` 注释掉了后面的密码验证——你不需要密码就能登录任何账户。

你在 05 学的 XSS，攻击的也是表单。一个搜索框，用户输入的不是搜索关键词，而是一段 JavaScript 代码：

```
<script>alert('守夜者，我在。')</script>
```

如果后端没有过滤用户输入，这段代码会在搜索结果页面上被执行——每个打开这个页面的人都会弹出一个写着“守夜者，我在。”的弹窗。

你以后挖漏洞的时候，第一件事就是找表单。登录框、搜索框、评论区、留言板、文件上传入口——每一个 `<form>` 标签、每一个 `<input>` 标签都是一个潜在的漏洞入口。你在 HTML 源码里看到一个 `<input type="text" name="username">`，你就知道：“这里可以输入数据。如果后端没有过滤这段数据——我就进来了。”

Kate 在 VS Code 里写了一个简单的登录表单，然后盯着 `<input type="text" name="username">` 那一行看了很久。她说：“所以这行代码——看起来只是让用户输入用户名的——如果后端没有做好过滤，这行代码就是攻击者的入口。SQL 注入从这里进去，XSS 也从这里进去。”

“对。HTML 是网页的骨架，表单是骨架上的关节。关节越灵活，你越容易掰开它。但关节越脆弱，攻击者越容易打断它。你以后学后端开发（06-2）的时候，会学到怎么加固这些关节——怎么过滤用户输入、怎么转义特殊字符、怎么验

证数据类型。但你要先知道关节长什么样，才知道怎么掰开它，也才知道怎么保护它。”

第三章：CSS——网页的皮肤

3.1 CSS 是什么？

- CSS 让网页从“白纸黑字”变成“有颜色、有布局、有动画”
- CSS 写在 HTML 的什么位置？内联、内部、外部——三种方式

3.2 CSS 的基本语法

- 选择器、属性、值——CSS 的核心三要素
- 常用属性：颜色（`color`）、背景（`background`）、字体大小（`font-size`）、间距（`margin`、`padding`）
- 亲手写一段 CSS，让刚才的网页变好看

3.3 CSS 和 HTML 的关系

- HTML 定义“有什么”（骨架），CSS 定义“长什么样”（皮肤）
- 为什么要分开？因为你可以给同一个 HTML 换不同的 CSS，网页就完全换了一张脸

第四章：JavaScript——网页的大脑

4.1 JavaScript 是什么？

- JavaScript 让网页“动起来”——点击按钮弹出提示、输入框实时验证、页面不刷新就能加载新内容
- JavaScript 运行在浏览器里，不是服务器上

4.2 JavaScript 的基本语法

- 变量、函数、事件——三个概念就能写出能用的代码
- 亲手写一段 JS：点击按钮，弹出一个“守夜者，我在。”的弹窗
- 这个弹窗你在 05 第二章亲手验证 XSS 时弹过——现在你知道它是怎么弹出来的了

4.3 JavaScript 被恶意利用：XSS 的原理

- 你在 05 学的 XSS，本质上就是“让别人的浏览器执行我写的 JavaScript”
- 理解了 JS，你就理解了 XSS 为什么能偷 Cookie、劫持会话、重定向页面

第五章：HTTP 协议——Web 的语言

5.1 HTTP 是什么？

- 你在 05 第四章学过 HTTP——这里用更直观的方式重温
- HTTP 请求的三个部分：请求行、请求头、请求体
- HTTP 响应的三个部分：状态行、响应头、响应体

5.2 常见的 HTTP 方法

- GET——请求数据，参数拼在 URL 后面
- POST——提交数据，参数放在请求体里
- 为什么渗透测试中要看请求方法？因为 GET 参数暴露在 URL 里，更容易被篡改

5.3 常见的 HTTP 状态码

- 200 OK——正常
- 301/302——重定向
- 403——禁止访问
- 404——找不到
- 500——服务器内部错误
- 你以后扫描目录、爆破接口的时候，就是根据这些状态码判断有没有找到隐藏的入口

5.4 Cookie 和 Session

- HTTP 是无状态的——每次请求都是独立的，服务器不知道你是谁
- Cookie 和 Session 让服务器“记住”你——登录一次，下次不用重新输密码
- XSS 攻击最想偷的是什么？你的 Cookie

第六章：用开发者工具透视网页——你的第一把“透视眼”

6.1 从 HTML 到像素——浏览器做了什么？

- 浏览器收到 HTML → 解析标签 → 构建 DOM 树 → 加载 CSS → 构建渲染树 → 绘制到屏幕
- 为什么要知道这个过程？因为 XSS 攻击本质上是篡改了 DOM 树——你在网页里插入了一段不该存在的 HTML

6.2 浏览器的开发者工具——你的第一把“透视眼”

- 按 F12 打开开发者工具
- Elements 标签——查看和修改网页的 HTML 和 CSS
- Console 标签——执行 JavaScript 代码，查看报错信息
- Network 标签——监控所有网络请求，看到每个请求的 URL、方法、状态码、响应时间
- 你在 05 第四章用 Wireshark 抓包，现在你用 Network 标签抓包——同一个原理，不同的工具

第七章：总结与展望

7.1 你学会了什么？

- 一个网页由 HTML（骨架）、CSS（皮肤）、JavaScript（大脑）三层组成
- 浏览器和服务器通过 HTTP 协议通信——每次请求独立，Cookie 和 Session 维持状态
- 你能用 F12 透视网页的内部结构

7.2 这些知识在渗透测试中怎么用？

- HTML 表单 → SQL 注入入口

- JavaScript → XSS 攻击的武器
- Cookie → XSS 攻击的目标
- HTTP 请求 → 抓包分析、Burp Suite 改包重放
- 状态码 → 敏感目录扫描

7.3 下一步学什么？

- 06-2 《Flask Web 开发》——用 Python 搭建你自己的 Web 应用，你的“自家靶场”
- 06-3 《用 Python 写安全工具（基础）》——从爬虫到 GUI，把你的脚本变成工具
- 06-4 《用 Python 写安全工具（进阶）》——端口扫描器、弱口令爆破、日志分析脚本